Soft Contract Verification for Higher-order Languages

Ling Zheng
Northeastern University

Contract Checking in ACL2s

```
(definec div (n :nat m :pos) :nat
  (floor n m))

(definec map-div (n :nat ms :pos-list) :nat-list
  (cond ((null ms) nil)
        (t (cons (div n (car ms)) (map-div n (cdr ms))))))

(definec main (n :nat) :nat-list
  (map-div n '(42 0)))
```

Contract Checking in ACL2s

```
(definec div (n :nat m :pos) :nat
  (floor n m))

(definec map-div (n :nat ms :pos-list) :nat-list
  (cond ((null ms) nil)
        (t (cons (div n (car ms)) (map-div n (cdr ms))))))

(definec main (n :nat) :nat-list
  (map-div n '(42 0)))
```

⇒ **static contract violation!**

Contract Checking in ACL2s

```
(definec div (n :nat m :pos) :nat
  (floor n m))

(definec map-div (n :nat ms :pos-list) :nat-list
  (cond ((null ms) nil)
        (t (cons (div n (car ms)) (map-div n (cdr ms))))))

(definec main (n :nat) :nat-list
  (map-div n '(42 0)))
```

⇒ **static contract violation!**

- **Why?** The list `'(42 0)` contains `0`, which violates the `:pos` constraint in `div`.

Contract Checking in ACL2s

Contract Checking in ACL2s

- Return value violates the return contract.

Contract Checking in ACL2s

- Return value violates the return contract.

```
(definec map-div (n :nat ms :pos-list) :nat
  (cond ((null ms) nil)
        (t (cons (div n (car ms)) (map-div n (cdr ms))))))
```

Contract Checking in ACL2s

- Return value violates the return contract.

```
(definec map-div (n :nat ms :pos-list) :nat
  (cond ((null ms) nil)
        (t (cons (div n (car ms)) (map-div n (cdr ms))))))
```

- Input arguments violate the input contracts.

Contract Checking in ACL2s

- Return value violates the return contract.

```
(definec map-div (n :nat ms :pos-list) :nat
  (cond ((null ms) nil)
        (t (cons (div n (car ms)) (map-div n (cdr ms))))))
```

- Input arguments violate the input contracts.

```
(definec main (n :nat) :nat-list
  (map-div n '(42 0)))
```

Contract Checking in Higher-order Languages

```
(div : nat? (>/c 0) -> nat?)

(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))

(main : nat? -> (listof nat?))
(define (main n)
  (map (λ (m) (div n m)) '(42 0)))
```

Contract Checking in Higher-order Languages

```
(div : nat? (>/c 0) -> nat?)

(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))

(main : nat? -> (listof nat?))
(define (main n)
  (map (λ (m) (div n m)) '(42 0)))
```

Contract Checking in Higher-order Languages

```
(div : nat? (>/c 0) -> nat?)

(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))

(main : nat? -> (listof nat?))
(define (main n)
  (map (λ (m) (div n m)) '(42 0)))
```

Contract Checking in Higher-order Languages

```
(div : nat? (>/c 0) -> nat?)

(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))

(main : nat? -> (listof nat?))
(define (main n)
  (map (λ (m) (div n m)) '(42 0)))
```

Contract Checking in Higher-order Languages

```
(div : nat? (>/c 0) -> nat?)

(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))

(main : nat? -> (listof nat?))
(define (main n)
  (map (λ (m) (div n m)) '(42 0)))

(main 42)
```

Contract Checking in Higher-order Languages

```
(div : nat? (>/c 0) -> nat?)

(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))

(main : nat? -> (listof nat?))
(define (main n)
  (map (λ (m) (div n m)) '(42 0)))

(main 42)
```

⇒ **dynamic contract violation!**

Contract Checking in Higher-order Languages

- Extra runtime monitoring imposes significant overhead.

Contract Checking in Higher-order Languages

- Extra runtime monitoring imposes significant overhead.
- We can only identify problems by executing the program.

Contract Checking in Higher-order Languages

- Extra runtime monitoring imposes significant overhead.
- We can only identify problems by executing the program.

We need static contract checking for higher-order languages!

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution

- Abstract Interpretation

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution
  - Pros: Bug-finding
- Abstract Interpretation

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution
  - Pros: Bug-finding
  - Cons: Non-termination
- Abstract Interpretation

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution
  - Pros: Bug-finding
  - Cons: Non-termination
- Abstract Interpretation
  - Pros: **Verification**

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution
  - Pros: Bug-finding
  - Cons: Non-termination
- Abstract Interpretation
  - Pros: **Verification**
  - Cons: False positives

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution
  - Pros: Bug-finding
  - Cons: Non-termination
- Abstract Interpretation
  - Pros: **Verification**
  - Cons: False positives

SCV can statically prove either **complete or partial** contract correctness.

Soft Contract Verification (Phúc C. Nguyen, Sam Tobin-Hochstadt, and David Van Horn. 2014)

- Symbolic Execution
  - Pros: Bug-finding
  - Cons: Non-termination
- Abstract Interpretation
  - Pros: **Verification**
  - Cons: False positives

SCV can statically prove either **complete or partial** contract correctness.

SCV can prove and remove 99% (2329/2335) of runtime contracts, resulting in a speed up $5x - 5000x$!

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

```
(- ({even? -> even?} (+ {odd?} 1)) 1)
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

$f \mapsto$ `{even? -> even?}`
$n \mapsto$ `{odd?}`

```
(- ({even? -> even?} (+ {odd?} 1)) 1)
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

```
(- ({even? -> even?} {even?}) 1)
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

```
(- ({even? -> even?} {even?}) 1)
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

```
(- {even?} 1)
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

```
(- {even?} 1)
```

Higher-order Symbolic Reasoning

```
(e2o : (even? -> even?) -> (odd? -> odd?))
(define (e2o f)
  (λ (n) (- (f (+ n 1)) 1)))
```

f ↦ {even? -> even?}
n ↦ {odd?}

{odd?}

Flow Sensitive Reasoning

```
(f : (or/c int? str?) cons? -> int?)
(define (f x p)
  (cond [(and (int? x) (int? (car p))) (+ x (car p))]
        [(int? (car p)) (+ (str-len x) (car p))]
        [else 0]))
```

Flow Sensitive Reasoning

```
(f : (or/c int? str?) cons? -> int?)
(define (f x p)
  (cond [(and (int? x) (int? (car p))) (+ x (car p))]
        [(int? (car p)) (+ (str-len x) (car p))]
        [else 0]))

(cond [(and (int? {(or/c int? str?)}) (int? (car {cons?})))
       (+ {(or/c int? str?)} (car {cons?}))]
       ...)
```

Flow Sensitive Reasoning

```
(f : (or/c int? str?) cons? -> int?)
(define (f x p)
  (cond [(and (int? x) (int? (car p))) (+ x (car p))]
        [(int? (car p)) (+ (str-len x) (car p))]
        [else 0]))
```

$\mathtt{x} \mapsto L_1, \boxed{L_1 \mapsto \{\mathtt{(or/c\ int?\ \ str?)}\}}$

$\mathtt{p} \mapsto L_2, \boxed{L_2 \mapsto \{\mathtt{cons?}\}}$

```
(cond [(and (int? L1) (int? (car L2)))
       (+ L1 (car L2))]
       ...)
```

Flow Sensitive Reasoning

```
(f : (or/c int? str?) cons? -> int?)
(define (f x p)
  (cond [(and (int? x) (int? (car p))) (+ x (car p))]
        [(int? (car p)) (+ (str-len x) (car p))]
        [else 0]))
```

$\mathtt{x} \mapsto L_1, \boxed{L_1 \mapsto \{\mathtt{(or/c\ int?\ \ str?)}\}}$

$\mathtt{p} \mapsto L_2, \boxed{L_2 \mapsto \{\mathtt{cons?}\}}$

```
(cond [(and (int? L1) (int? (car L2)))
        (+ L1 (car L2))]
        ...)
```

Flow Sensitive Reasoning

```
(f : (or/c int? str?) cons? -> int?)
(define (f x p)
  (cond [(and (int? x) (int? (car p))) (+ x (car p))]
        [(int? (car p)) (+ (str-len x) (car p))]
        [else 0]))
```

$x \mapsto L_1, \boxed{L_1 \mapsto \{\texttt{(or/c int? str?)}\}}$

```
(if (int? L1)
    (+ L1 ...)
    ...)
```

$\boxed{L_1 \mapsto \{\texttt{int?}\}}$

$\boxed{L_1 \mapsto \{\texttt{str?}\}}$

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ (cons (<λ> m) (map <λ> ms'))

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ (cons (<λ> m) (map <λ> ms'))

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ (cons (<λ> m) '())

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ (cons (div 42 m) '())

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ (cons (div 42 m) '())

((and/c nat? (>/c 0)) m) $\Longrightarrow$ (div 42 m) $\rightarrow$ nat?

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ '()

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ (cons (div 42 m) '())

((and/c nat? (>/c 0)) m) $\Longrightarrow$ (div 42 m) $\rightarrow$ nat?

((and/c nat? (not (>/c 0))) m) $\Longrightarrow$ (div 42 m) $\rightarrow$ <blame>

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ {'()}

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ {(cons nat? '()), <blame>}

On-demand Summarization

```
(map : (any/c -> any/c) list? -> list?)
(define (map f xs)
  (cond ([(null xs) '()]
         [else (cons (f (car xs)) (map f (cdr xs)))])))
(div : nat? (>/c 0) -> nat?)
(main : (listof nat?) -> (listof nat?))
(define (main ms)
  (map (λ (m) (div 42 m)) ms))
```

(null ms) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ {'()}

(not (null ms)) $\Longrightarrow$ (map <λ> ms) $\rightarrow$ {<non-empty (listof nat?)>, <blame>}

SMT Solver-aided Reasoning

```
(define (f x g) (g (+ x 1)))

(h : [x : int?] -> [y : (>/c x)] -> (>/c y))
(define (h x) ...)

(main : int? -> (>/c 0))
(define (main n) (if (≥ n 0) (f n (h n)) 1))
```

$n \mapsto$ { int?, (≥/c 0) }
$n_1 \mapsto$ { int?, (=/c (+ n 1)) }
$n_2 \mapsto$ { int?, (>/c n_1) }

SMT Solver-aided Reasoning

$\texttt{n} \mapsto \{\ \texttt{int?},\ (\geq\texttt{/c}\ \texttt{0})\ \}$
$n_1 \mapsto \{\ \texttt{int?},\ (\texttt{=/c}\ (\texttt{+}\ \texttt{n}\ \texttt{1}))\ \}$
$n_2 \mapsto \{\ \texttt{int?},\ (\texttt{>/c}\ n_1)\ \}$

SMT Solver-aided Reasoning

$\mathtt{n} \mapsto \{\ \mathtt{int?},\ (\geq\mathtt{/c}\ \mathtt{0})\ \}$
$n_1 \mapsto \{\ \mathtt{int?},\ (\mathtt{=/c}\ (\mathtt{+}\ \mathtt{n}\ \mathtt{1}))\ \}$
$n_2 \mapsto \{\ \mathtt{int?},\ (\mathtt{>/c}\ n_1)\ \}$

```
(declare-const n Int)
(declare-const n1 Int)
(declare-const n2 Int)
(assert (>= n 0))
(assert (= n1 (+ n 1)))
(assert (> n2 n1))
(assert (not (> n2 0)))
(check-sat)
```

SMT Solver-aided Reasoning

$\texttt{n} \mapsto \{\ \texttt{int?},\ (\geq\texttt{/c}\ \texttt{0})\ \}$
$n_1 \mapsto \{\ \texttt{int?},\ (\texttt{=/c}\ (\texttt{+}\ \texttt{n}\ \texttt{1}))\ \}$
$n_2 \mapsto \{\ \texttt{int?},\ (\texttt{>/c}\ n_1)\ \}$

```
(declare-const n Int)
(declare-const n1 Int)
(declare-const n2 Int)
(assert (>= n 0))
(assert (= n1 (+ n 1)))
(assert (> n2 n1))
(assert (not (> n2 0)))
(check-sat)
```

$\Rightarrow$ **unsat**

SMT Solver-aided Reasoning

$\mathtt{n} \mapsto \{\ \mathtt{int?},\ (\geq\!/\mathtt{c}\ \ 0)\ \}$
$n_1 \mapsto \{\ \mathtt{int?},\ (=\!/\mathtt{c}\ \ (+\ \mathtt{n}\ \ 1))\ \}$
$n_2 \mapsto \{\ \mathtt{int?},\ (>\!/\mathtt{c}\ \ n_1)\ \}$

zheng.lingx@northeastern.edu

```
(declare-const n Int)
(declare-const n1 Int)
(declare-const n2 Int)
(assert (>= n 0))
(assert (= n1 (+ n 1)))
(assert (> n2 n1))
(assert (not (> n2 0)))
(check-sat)
```

$\Rightarrow$ **unsat**